

Brian Abbott

CSCI 2503 - Survey of Programming Languages
Georgia Southern University - Summer 2026

Assignment 6

Problem 1 - Reflection on Rust

Overall, I enjoyed working with Rust, even though it was easily the most demanding language I worked with this semester. Coming from Kotlin, Java, C++, and Python, I am used to languages that let me move quickly and either trust me to manage memory myself or hand that responsibility to a garbage collector. Rust takes a different stance: it forces me to be explicit about ownership up front, and in exchange, it makes an unusual promise - if the program compiles, an entire class of memory and concurrency bugs simply cannot occur. That trade is what won me over. Once I got a feel for what the compiler expected, the experience became genuinely rewarding. C is built on the philosophy that the programmer knows what they are doing. That sounds reasonable, but the vast majority of serious security vulnerabilities in the Linux kernel are memory-safety bugs - use-after-frees and buffer overflows - and the people doing kernel development are among the most capable systems programmers in the world. If even they can't reliably avoid these bugs in C, the problem is the model, not the programmer. That is exactly the gap Rust's ownership system is built to close.

The parts that felt most straightforward were those where my prior experience carried over. The basic syntax - functions, structs, control flow, and pattern matching - was easy to pick up given my previous coursework in C++. Cargo was a real highlight: adding dependencies, building, testing, and formatting all live in a single, consistent tool, which is far less painful than wrestling with build configuration in other ecosystems. I also appreciated how Rust handles the absence of a value. Instead of null, you get Option and Result, and the ? operator makes propagating errors clean and readable. Enums that carry data, combined with exhaustive match statements, felt like a more honest way to model state than I am used to, and the compiler refusing to let me forget a case caught real mistakes before they ran.

The parts I had the most trouble with centered on the borrow checker, which I suspect is true for almost everyone learning Rust. Ownership made sense in the abstract, but applying it in practice - passing references around, dealing with mutable versus immutable borrows, and especially adding lifetime annotations to structs - was where I repeatedly got stuck. There were moments where I knew exactly what I wanted the code to do but could not immediately convince the compiler it was safe. Smart pointers like Box, Rc, and RefCell added another layer I had to learn before some patterns I take for granted in garbage-collected languages would work at all. Traits and

generics with bounds were powerful but got dense quickly, and I only scratched the surface of async Rust, which felt like a steep cliff of its own.

My overall impression is that Rust is a serious, well-designed language that asks a lot of you and gives a lot back. The learning curve is real and front-loaded - the difficulty is greatest early, while you are still fighting the borrow checker - but the compiler is unusually good at teaching. Its error messages often explain not just what is wrong but how to fix it, with feedback specific enough that it frequently does the job I would otherwise turn to an LLM for - and unlike an LLM, it is never guessing. I do not think Rust is the right tool for every job; for much of the application and backend work I do day-to-day, I will still reach for Kotlin. But its relevance isn't abstract to me: as an Android developer, I know Rust is increasingly used for the system-level layer beneath the app - Google has been writing parts of Android's OS in it - so it's a skill I've tucked away as directly useful, not just admirable. The discipline it enforces has already changed how I think about ownership and mutation in the other languages I use.

Problem 2 - GUI Programming Reflection

Which framework did I used and why? For the GUI portion, I built a new native Android application in Kotlin using Jetpack Compose. I chose this combination for a few reasons. First of all, I am going to specialize in Kotlin and Java on the JVM, and Compose is the modern, declarative UI toolkit that pairs with modern Android development, so the assignment was a chance to deepen a skill I actually use rather than learning a throwaway framework. Second, Compose fits the assignment requirements cleanly: it is a true GUI framework with first-class event handling - its controls expose callbacks such as `onClick` and `onValueChange`, and it ships with a full set of Material 3 components (text fields, buttons, sliders, switches, and more) that look polished without much manual styling. Third, it suits the “imagine a non-technical user” part of the brief well, because Material components come with sensible defaults for spacing, touch targets, and accessibility. The application I coded is a simple AUTH workflow. At work, I had these old designs in Zeplin that hadn't been coded before. I coded these in Jetpack Compose, keeping in line with the assignment's spirit that they couldn't be old work.

The application takes input from the user through several interactive controls and, in response to an event the user initiates, performs a computation and shows a visible result on screen. It uses more than the required minimum of four unique controls, and each control produces a clear, visible response when the user interacts with it.

Was working in this language easy or hard? It was a mix of both. The language itself made things easier: Kotlin is concise, its null safety eliminated a category of bugs before they could happen, and features like data classes and lambdas kept the code compact.

Because I already know the language, I could focus on the interface rather than on syntax. The harder part was the declarative mental model. Compose does not work like the imperative UI code where you grab a control by reference and mutate it directly; instead, you describe what the UI should look like for a given state, and the framework recomposes the screen whenever that state changes. Getting comfortable with state - holding it with `remember` and `mutableStateOf`, hoisting it so the right components can read and update it, and reasoning about what triggers recomposition - was the main learning curve. Once that clicked, building and changing the interface became fast and predictable, and the live previews in Android Studio made it easy to iterate. On balance, I found it easier than the equivalent work would have been in a framework where I did not already have a language foundation like Swift.

I am not married to native development, but I have chosen it for every app I have built for work, rather than depending on a cross-platform framework. With something like React Native, when you need a platform capability - photo support, or background jobs through `WorkManager` - you typically install an npm module that wraps the underlying native code, instead of calling that API directly the way I can in native Android. If an app were truly just forms, I would be open to using React Native. The bottom line, though, is that I will use Jetpack Compose for all new native apps going forward. I won't rewrite my existing XML-based views in Compose, but all new UI will be built with Compose.

Finally, this project has me questioning at what level I need to learn things to be a good developer. For academia, I understand that I write the code myself. But AI has gotten genuinely good at building views now. I attended a tech talk where a student said, "You just vibe code the UI," and it stuck with me. Coding a view was always easy when you already had a design that looked good - the hard part, and the thing most people couldn't do, was producing a visually appealing design in the first place. What has changed is that AI is now capable of both: creating the design and writing the code for it.

When I started CSCI 1301, AI tools weren't very good. I had Codex fix a bug I had introduced into a WordPress plugin, but those tools couldn't handle the Android development I was doing, even though my apps are relatively simple. Now I can vibe-code new features into Android and debug them with Logcat. That works because I already know how to debug Android apps, and because these tools run in the command line, which fits my workflow - together they have sped me up considerably. What I am left wondering is where I still need to learn things to be as good a developer as I can be.

The recommended architecture for Android Development when I first started was Model View Viewmodel (MVVM). Model Viewmodel addresses a lot of separation of concerns. Model View Viewmodel also can contain a repository, or you can omit it depending on

how your application functions. When I first started learning Android Development, I coded the XML view files.

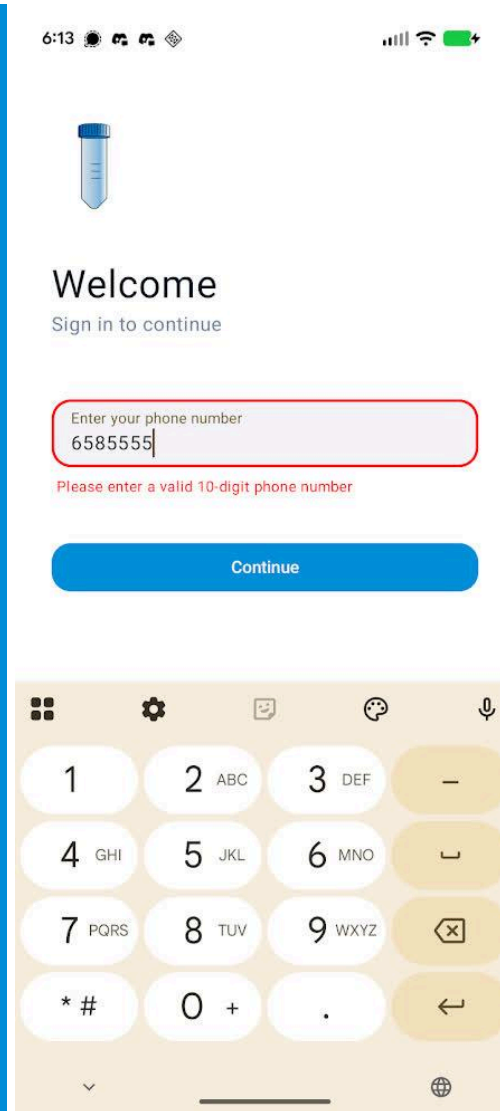
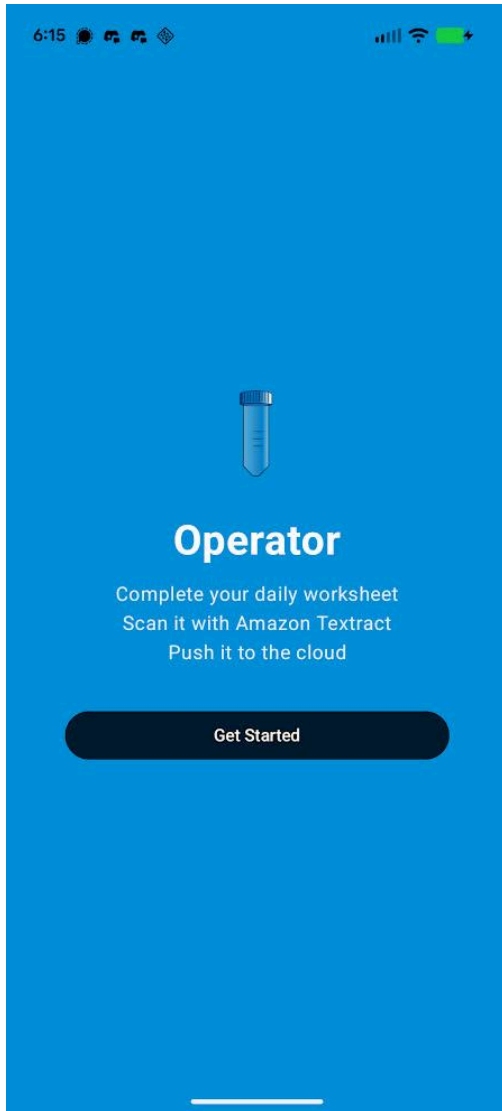
The application I coded (screenshots below) is a login authentication based on designs, as well as a dashboard and cardview that have been coded by a coworker. Operators have to enter a lot of wastewater data on a daily basis. It's entered into Excel almost daily, and the Excel spreadsheet they use is large. It's so large that it's not practical for us to generate it with code; we use an existing spreadsheet.

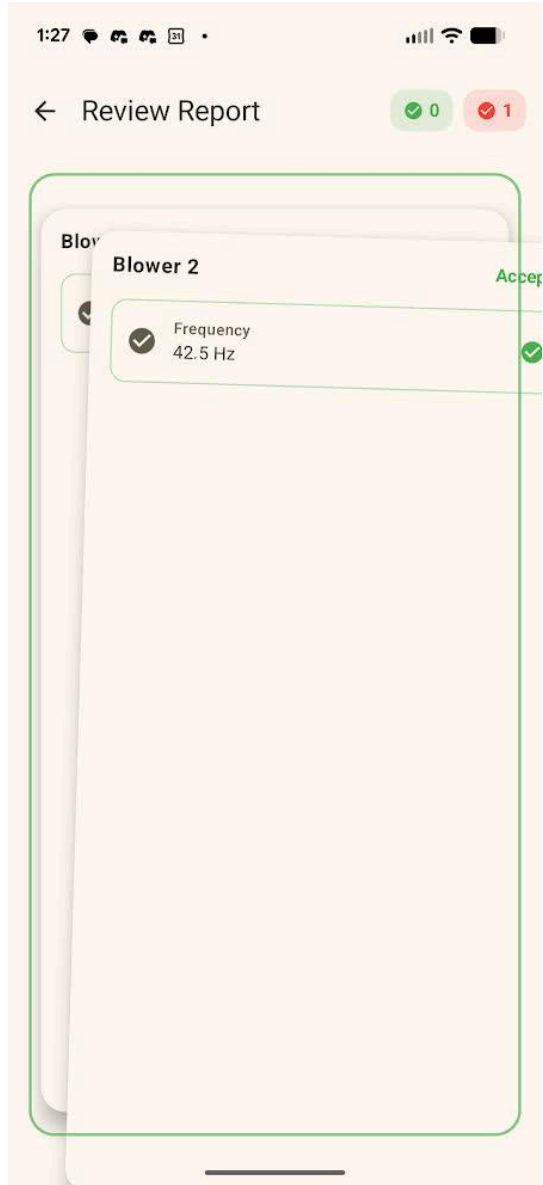
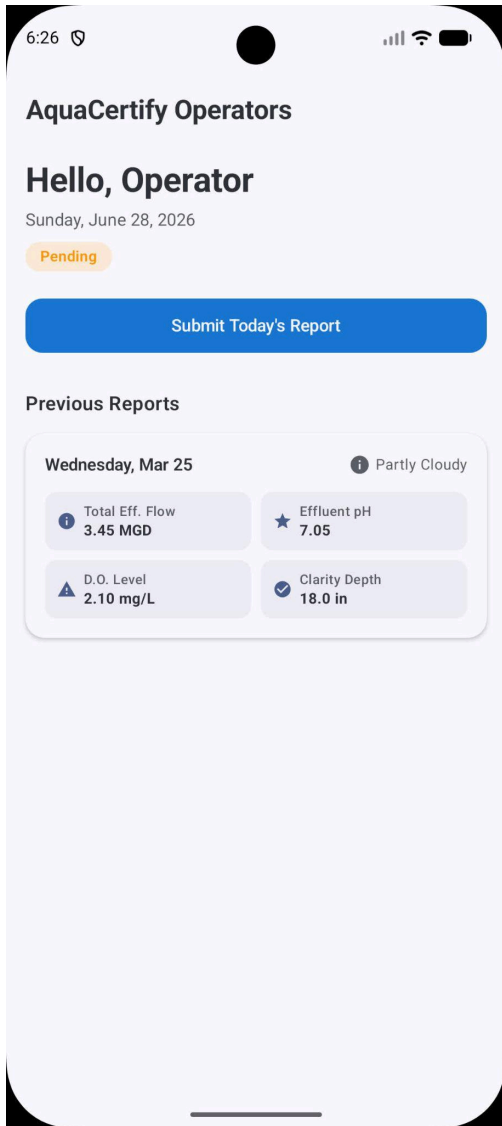
The data that is entered is written down when people are wearing latex gloves. My coworker, Walker had the brilliant idea to use OCR to scan the sheet and to use Amazon Textract to extract the data from tables (it uses OCR and scans the data in a structured format, based on tables or forms). So once the data has been processed, the user will swipe right to accept the data, or swipe left if it needs to be entered again.

In conclusion, I think that picking up different languages is kind of like learning different genres on guitar. It can help you get out of a rut, and it can also make learning newer genres easier.

Problem 3 — Application Screenshot

A screenshot of the running application is shown below.





1:27



← Review Report

✓ 2

✗ 2

ical Tanks

Reject

Phos Sulphate
850 gal



PAC Split
620 gal



PAC
100 gal

